🛠 Phase 25: "How Did You Create It?" — BA Methodology Guide

Document Code: HOW-DOC-001

Project: Duplicate Bin Ecosystem

Purpose: Deep-Dive Explanations of BA Deliverable Creation & Methodologies

Format: Objective | Methodology Applied | Project-Specific Implementation | Verifiable Artifact

1. How Did You Gather Requirements?

- **Methodology Applied:** Multi-channel elicitation combining stakeholder interviews, codebase reverse-engineering, document analysis, and competitive product benchmarking.
- **Project-Specific Execution:**

  "I initiated requirements elicitation by conducting discovery sessions with two primary user segments: creative media prosumers (photographers/video editors) and corporate IT administrators. I complemented user interviews with a rigorous technical reverse-engineering audit of our existing Python codebase (`scripts/duplicate_bin.py`), cataloging all 30 advanced features. To address enterprise needs, I analyzed storage cost pain points on cloud platforms (Google Workspace, Dropbox) and drafted our API requirements for FastAPI SaaS pairing."
- **Verifiable Artifact:** 01_PROJECT_INVENTORY.md, STAKEHOLDER_INTERVIEW_QUESTIONS.md.

2. How Did You Identify Stakeholders?

- **Methodology Applied:** Stakeholder Onion Diagramming and Mendelow's Power-Interest Matrix.
- **Project-Specific Execution:**

  "I categorized stakeholders across three concentric rings: External Customers (photographers, digital agency owners), Internal Operational Stakeholders (corporate IT administrators, security auditors), and Internal Delivery Team (Product Owner, Lead Architect, QA Lead). By plotting them on Mendelow's Matrix, I identified that Security Auditors held High Power despite Low Interest; therefore, I proactively authored formal Zero-Trust architecture proofs to satisfy compliance requirements early."

- **Verifiable Artifact:** 03_STAKEHOLDER_ANALYSIS.md.

3. How Did You Create the BRD?

- **Methodology Applied:** PMI Standard Business Requirements Document framework.
- **Project-Specific Execution:**

  "I structured the BRD to clearly establish the business problem before touching any technical specifications. I detailed the financial impact of unmanaged storage sprawl (15% to 35% wasted space), defined measurable business objectives (OBJ-01 through OBJ-05), delineated in-scope versus out-of-scope boundaries, and established the core business rules—such as the Master Preservation Rule (BRULE-01). Every requirement was linked to an executive sign-off matrix."
- **Verifiable Artifact:** 06_BRD/BRD.md.

4. How Did You Create the FRD?

- **Methodology Applied:** IEEE 830 Specification for Software Requirements.
- **Project-Specific Execution:**

  "For each major capability—such as two-pass size pre-filtering, Union-Find clustering, and 6-digit device pairing—I authored comprehensive functional specifications. Each feature was documented with its Actor, Preconditions, Trigger, Main Success Flow, Alternative Flows, Exception Conditions, and Business Rules. I concluded each specification with unambiguous Gherkin acceptance criteria to facilitate automated test scripting."
- **Verifiable Artifact:** 07_FRD/FRD.md.

5. How Did You Create the PRD?

- **Methodology Applied:** Modern Agile Product Management PRD Framework.
- **Project-Specific Execution:**

  "I focused on the Product Vision, User Personas ('Elena the Photographer' and 'Marcus the IT Lead'), Product Goals vs. Non-Goals, and Go-to-Market Release Strategy. I used the MoSCoW framework to

Product Goals vs. Non-Goals, and Go-to-Market Release Strategy. I used the MoSCoW framework to rigorously prioritize features, providing clear business and technical justifications for why capabilities like Cryptographic Hashing and Safe Archiving were 'Must Haves', while direct cloud API crawling was classified as 'Won't Have' for the current release."

- **Verifiable Artifact:** 08_PRD/PRD.md.

6. How Did You Create User Stories & Define Acceptance Criteria?

- **Methodology Applied:** Mike Cohn Agile User Story Syntax (`As a... I want... So that...`) with Gherkin BDD (`Given-When-Then`).
- **Project-Specific Execution:**

  "I grouped 11 user stories under 5 business epics. For each story, I quantified the underlying business value. In US-004 (Selection Heuristics), I defined the acceptance criteria to explicitly enforce that exactly one master file is preserved, ensuring developers had precise boolean logic to write unit tests against."
- **Verifiable Artifact:** 09_USER_STORIES/USER_STORIES.md.

7. How Did You Create Process Flows & Perform Gap Analysis?

- **Methodology Applied:** BPMN 2.0 Process Modeling & As-Is / To-Be Gap Analysis.
- **Project-Specific Execution:**

  "I mapped the legacy As-Is process—highlighting the extreme manual labor of ocular file comparisons and the risk of permanent accidental deletion. I then modeled the To-Be process in Mermaid diagrams, showing how our two-pass pre-filtering, SQLite persistent caching, and safe archive rollback eliminated these pain points. The resulting Gap Analysis clearly articulated the operational leap from 100 files/hour manually to 10,000 files in 18.7 seconds."
- **Verifiable Artifact:** 04_PROCESS_ANALYSIS.md, 11_PROCESS_FLOWS/PROCESS_FLOWS.md.

8. How Did You Create the RTM?

- **Methodology Applied:** Bidirectional Requirements Traceability Matrix (ISO/IEC 29148).
- **Project-Specific Execution:**

- **Project-Specific Execution:**

  "I built a tabular matrix linking all 24 formal requirements through their entire lifecycle: Business Requirement $\rightarrow$ Functional Requirement $\rightarrow$ User Story $\rightarrow$ Acceptance Criteria $\rightarrow$ Source Code Component $\rightarrow$ Automated Test $\rightarrow$ UAT Test Case. This gave our engineering and executive teams 100% confidence that no requirements were orphaned and that every test case validated a real business need."

- **Verifiable Artifact:** 18_RTM/RTM.md.

9. How Did You Prioritize Requirements?

- **Methodology Applied:** MoSCoW Prioritization coupled with Value vs. Complexity Matrix.
- **Project-Specific Execution:**

  "I categorized requirements into Must Have, Should Have, Could Have, and Won't Have. 'Must Have' was reserved strictly for features critical to core viability and user safety (cryptographic integrity, zero-loss quarantine archive). Features that enhanced ergonomics (drag-and-drop, side-by-side previews) were classified as 'Should Have'. High-effort external integrations (Google Drive API crawling) were categorized as 'Won't Have' to protect our release timeline."

- **Verifiable Artifact:** 08_PRD/PRD.md#L4.

10. How Did You Handle Changing Requirements?

- **Methodology Applied:** Formal Change Control Board (CCB) Governance.
- **Project-Specific Execution:**

  "When Product requested increasing our free introductory credit tier from 5 GB to 10 GB, I logged Change Request CR-002. I assessed the impact across our SQLAlchemy database model, API serialization, and financial onboarding funnel. I presented this to the Change Control Board, obtained formal sign-off, and updated our BRD, FRD, and test suites accordingly."

- **Verifiable Artifact:** 19_CHANGE_MANAGEMENT/CHANGE_REQUEST_LOG.md.

11. How Did You Perform UAT & Validate Project Success?

- **Methodology Applied:** Structured User Acceptance Testing with Business Personas & Entry/Exit Criteria.
- **Project-Specific Execution:**

  "I authored 10 comprehensive business test cases and facilitated hands-on testing with representative creative and IT stakeholders. We validated critical workflows, such as cold scanning 1,000 files, executing 1-click restore (`Ctrl+Z`), and verifying that cloud credit balances were decremented accurately upon scan completion. Project success was validated against our original KPIs: 100% data recovery, 87% faster rescans, and 0 critical defects."
- **Verifiable Artifact:** 17_UAT/UAT_PLAN.md, 17_UAT/UAT_TEST_CASES.md.